Sobre árboles de decisión

Comenzaremos revisando algunos conceptos básicos sobre el funcionamiento de los árboles de decisión. A medida que avancemos en las clases, exploraremos sus diversas características y desafíos, con el objetivo de comprender por qué, en problemas de modelado de datos tabulares, siguen siendo el estado del arte, superando incluso a los modelos más complejos de redes neuronales.

Recomendamos chusmear esta hermosa visualización de cómo funciona un árbol de decisión.

Cómo funciona un árbol de decisión

Y luego empezaremos a trabajar con nuestro conjunto de datos para *tratar* de modelar uno. Para eso, debemos iniciar el entorno

Setup

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, plot_tree,  _tree
```

Variables importantes

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
dataset_path = '/content/drive/MyDrive/DMEyF/2026/notebooks/data/'
dataset_file = 'competencia_01.csv'

ganancia_acierto = 1_072_500
costo_estimulo = 27_500
```

Cargamos el **dataset**, vamos a trabajar en esta ocasión solo con un periodo: **202104**.

Algo que va a generar algo de confusión es que no separemos con **Train / Test** en esta ocasión.

NO ENTRE EN PÁNICO, en esta cursada se le dedicará mucho tiempo a entender y combatir el problema del overfitting. Pero no hoy.

```
import os
print(os.path.exists('/content/drive/MyDrive/DMEyF/2026/notebooks/data/competencia_01.csv'))
```

```
True
```

```
import os

# Buscamos el archivo en todo tu Drive montado
print("Buscando el archivo competencia_01.csv...")
ruta_encontrada = None

for raiz, carpetas, archivos in os.walk('/content/drive/MyDrive/'):
    if 'competencia_01.csv' in archivos:
        ruta_encontrada = os.path.join(raiz, 'competencia_01.csv')
        print(f"¡ENCONTRADO! La ruta exacta es: {ruta_encontrada}")
        break

if not ruta_encontrada:
    print("No se encontró el archivo. Asegurate de que esté subido a tu Google Drive y que el nombre sea exactamente 'compe
```

```
Buscando el archivo competencia_01.csv...
¡ENCONTRADO! La ruta exacta es: /content/drive/MyDrive/DMEyF/2026/notebooks/data/competencia_01.csv
```

```
data = pd.read_csv(dataset_path+dataset_file)
data = data[data['foto_mes'] == 202104]

X = data.drop('clase_ternaria', axis=1)
y = data['clase_ternaria']
```

Veamos que porcentaje ocupa de nuestro dataset, el target que queremos predecir

```
data['clase_ternaria'].value_counts(normalize=True)*100
```

clase_ternaria	proportion
CONTINUA	98.718797
BAJA+2	0.695108
BAJA+1	0.586095

dtype: float64

- **¿Había trabajado con un target tan poco representativo?**
- **¿Cree que esto es algo común o tan solo una rareza?**

Primer árbol

Con nuestros datos ya cargados en **memoria**, podemos dedicarnos a generar nuestro primer árbol

Párametros

(gentileza de chat-gpt en un 90%)

El **DecisionTreeClassifier** en scikit-learn tiene varios parámetros que controlan cómo se construye el árbol de decisión. A continuación te explico los más importantes:

1. **criterion**: Este parámetro define la función que se usa para medir la calidad de una división (split). Las opciones principales son:
   - **gini** (predeterminado): Utiliza el índice de Gini, que mide la impureza de las particiones. Un valor de Gini más bajo significa que una partición es más pura.
   - **entropy**: Utiliza la entropía de la información, relacionada con el concepto de ganancia de información. Se enfoca en reducir la incertidumbre en las particiones.
2. **max_depth**: Controla la profundidad máxima del árbol.
3. **min_samples_split**: Especifica el número mínimo de muestras que un nodo debe tener para poder dividirse.
4. **min_samples_leaf**: Define el número mínimo de muestras que debe tener una hoja (nodo final).
5. **max_features**: Indica el número máximo de características que el modelo debe considerar para hacer la mejor división.
6. **max_leaf_nodes**: Especifica el número máximo de nodos hoja que el árbol puede tener.
7. **random_state**: Fija la semilla utilizada por el generador de números aleatorios, lo que garantiza que los resultados del entrenamiento sean reproducibles.
8. **ccp_alpha**: Parámetro de complejidad de poda mínima, utilizado para la poda de árboles de decisión después de que se hayan construido. Un valor más alto lleva a árboles más pequeños.

EN LA CURSADA CREEMOS QUE LA INTELIGENCIA DEL ALUMNO CORRELACIONA DE FORMA NEGATIVA A VALOR QUE SETEA ESTE PARÁMETRO

Queda de tarea para el alumnado jugar con algunas parametrías. Usaremos a continuación una simple

Modelo

```
model = DecisionTreeClassifier(criterion='gini',
                               random_state=17,
                               min_samples_split=80,
                               min_samples_leaf=1,
                               max_depth=5)

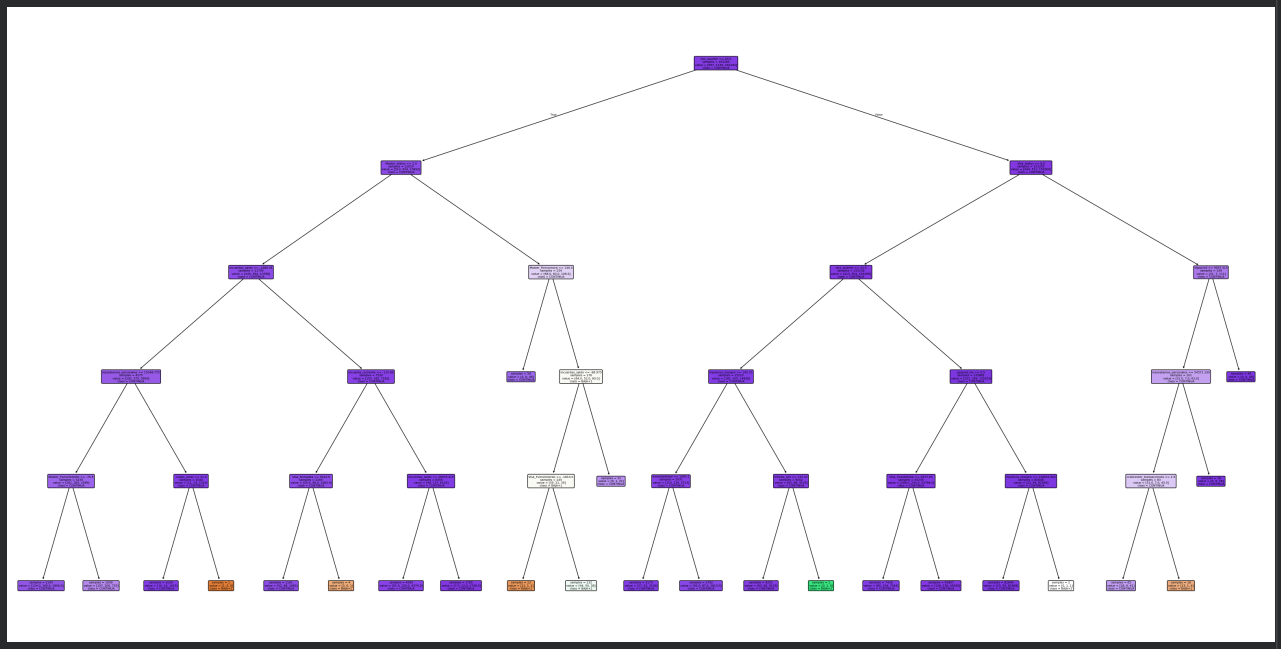
model.fit(X, y)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=5, min_samples_split=80, random_state=17)
```

Visualización

Todo es mucho más bonito si tiene colores

```
plt.figure(figsize=(40,20))
plot_tree(model, feature_names=X.columns, filled=True, class_names=model.classes_, rounded=True, impurity=False) #,  propor
plt.show()
```



Midiendo el modelo

Mirando tan solo la visualización

- **¿Qué tan bien performó el modelo?**
- **¿Qué tan bien detectó los CONTINUA?**
- **¿Qué tan bien detectó los BAJA+2?**

Para meter manos en la masa, vamos a llevar las hojas del modelo a una tabla, para esto haremos uso de una función auxiliar

```
def get_leaf_info(tree):
    tree_ = tree.tree_
    class_names = tree.classes_
    leaf_info = []
    for i in range(tree_.node_count):
        if tree_.children_left[i] == _tree.TREE_LEAF:
            class_counts = tree_.value[i][0] * int(tree_.n_node_samples[i])
            predicted_class_index = class_counts.argmax()
            predicted_class = class_names[predicted_class_index]
            row = {
                'Node': i,
                'Samples': int(tree_.n_node_samples[i]),
                'Predicted Class': predicted_class
            }
            for j, class_name in enumerate(class_names):
                row[class_name] = int(class_counts[j])
            leaf_info.append(row)
    return pd.DataFrame(leaf_info)
```

```
leaf_df = get_leaf_info(model)
leaf_df
```

	Node	Samples	Predicted Class	BAJA+1	BAJA+2	CONTINUA
0	5	2145	CONTINUA	124	165	1856
1	6	1090	CONTINUA	157	200	733
2	8	1039	CONTINUA	10	14	1015
3	9	1	BAJA+1	1	0	0
4	12	1161	CONTINUA	52	48	1061
5	13	4	BAJA+1	3	0	1
6	15	4590	CONTINUA	91	125	4374
7	16	1765	CONTINUA	7	12	1746
8	18	58	CONTINUA	4	8	46
9	21	13	BAJA+1	11	1	1
10	22	132	BAJA+2	44	50	38
11	23	31	CONTINUA	9	1	21
12	28	3179	CONTINUA	27	52	3100
13	29	2792	CONTINUA	91	87	2613
14	31	9251	CONTINUA	63	65	9123
15	32	1	BAJA+2	0	1	0
16	35	7428	CONTINUA	80	104	7244
17	36	45807	CONTINUA	128	139	45540
18	38	82646	CONTINUA	23	55	82568
19	39	2	BAJA+2	0	1	1
20	43	65	CONTINUA	18	6	41
21	44	18	BAJA+1	13	1	4
22	45	18	CONTINUA	0	0	18
23	46	48	CONTINUA	0	0	48

Próximos pasos: Generar código con `leaf_df` New interactive sheet

Ahora en el dataset **leaf_ds** tenemos cada nodo terminal (hoja).

Vemos que según el modelo, no deberíamos mandar ningún estímulo, dado que no detectó ningún **BAJA+2**. Sin embargo dudaremos de esto y veremos cuanta plata nos hubiera dejado cada hoja, si enviáramos a los clientes que cayeron en esta.

```
leaf_df["ganancia"] = ganancia_acierto*leaf_df["BAJA+2"] - costo_estimulo*leaf_df["BAJA+1"] + -costo_estimulo*leaf_df["CONT
leaf_df.sort_values("ganancia", ascending=False, inplace=True)
leaf_df
```

	Node	Samples	Predicted Class	BAJA+1	BAJA+2	CONTINUA	ganancia
1	6	1090	CONTINUA	157	200	733	190025000
0	5	2145	CONTINUA	124	165	1856	122512500
10	22	132	BAJA+2	44	50	38	51370000
4	12	1161	CONTINUA	52	48	1061	20872500
13	29	2792	CONTINUA	91	87	2613	18947500
6	15	4590	CONTINUA	91	125	4374	11275000
8	18	58	CONTINUA	4	8	46	7205000
20	43	65	CONTINUA	18	6	41	4812500
15	32	1	BAJA+2	0	1	0	1072500
19	39	2	BAJA+2	0	1	1	1045000
9	21	13	BAJA+1	11	1	1	742500
21	44	18	BAJA+1	13	1	4	605000
11	23	31	CONTINUA	9	1	21	247500
3	9	1	BAJA+1	1	0	0	-27500
5	13	4	BAJA+1	3	0	1	-110000
22	45	18	CONTINUA	0	0	18	-495000
23	46	48	CONTINUA	0	0	48	-1320000
2	8	1039	CONTINUA	10	14	1015	-13172500
12	28	3179	CONTINUA	27	52	3100	-30222500
7	16	1765	CONTINUA	7	12	1746	-35337500
16	35	7428	CONTINUA	80	104	7244	-89870000
14	31	9251	CONTINUA	63	65	9123	-182902500
17	36	45807	CONTINUA	128	139	45540	-1106792500
18	38	82646	CONTINUA	23	55	82568	-2212265000

Próximos pasos: Generar código con `leaf_df` New interactive sheet

Vaya! no son pocos las hojas que hubieran dado ganancia positiva, y todas a su vez estuvieron mal clasificadas por el modelo.

Veamos de calcular la probabilidad de **BAJA+2** por cada hoja

```
leaf_df["prob_baja_2"] = leaf_df["BAJA+2"]/leaf_df["Samples"]
leaf_df.sort_values("prob_baja_2", ascending=False, inplace=True)
leaf_df
```

	Node	Samples	Predicted Class	BAJA+1	BAJA+2	CONTINUA	ganancia	prob_baja_2
15	32	1	BAJA+2	0	1	0	1072500	1.000000
19	39	2	BAJA+2	0	1	1	1045000	0.500000
10	22	132	BAJA+2	44	50	38	51370000	0.378788
1	6	1090	CONTINUA	157	200	733	190025000	0.183486
8	18	58	CONTINUA	4	8	46	7205000	0.137931
20	43	65	CONTINUA	18	6	41	4812500	0.092308
0	5	2145	CONTINUA	124	165	1856	122512500	0.076923
9	21	13	BAJA+1	11	1	1	742500	0.076923
21	44	18	BAJA+1	13	1	4	605000	0.055556
4	12	1161	CONTINUA	52	48	1061	20872500	0.041344
11	23	31	CONTINUA	9	1	21	247500	0.032258
13	29	2792	CONTINUA	91	87	2613	18947500	0.031160
6	15	4590	CONTINUA	91	125	4374	11275000	0.027233
12	28	3179	CONTINUA	27	52	3100	-30222500	0.016357
16	35	7428	CONTINUA	80	104	7244	-89870000	0.014001
2	8	1039	CONTINUA	10	14	1015	-13172500	0.013474
14	31	9251	CONTINUA	63	65	9123	-182902500	0.007026
7	16	1765	CONTINUA	7	12	1746	-35337500	0.006799
17	36	45807	CONTINUA	128	139	45540	-1106792500	0.003034
18	38	82646	CONTINUA	23	55	82568	-2212265000	0.000665
22	45	18	CONTINUA	0	0	18	-495000	0.000000
5	13	4	BAJA+1	3	0	1	-110000	0.000000
3	9	1	BAJA+1	1	0	0	-27500	0.000000
23	46	48	CONTINUA	0	0	48	-1320000	0.000000

Próximos pasos: Generar código con `leaf_df` New interactive sheet

Finalmente calculemos la ganancia acumulada para cada posible punto de corte:

```
leaf_df['gan_acumulada'] = leaf_df['ganancia'].cumsum()
leaf_df
```

	Node	Samples	Predicted Class	BAJA+1	BAJA+2	CONTINUA	ganancia	prob_baja_2	gan_acumulada
15	32	1	BAJA+2	0	1	0	1072500	1.000000	1072500
19	39	2	BAJA+2	0	1	1	1045000	0.500000	2117500
10	22	132	BAJA+2	44	50	38	51370000	0.378788	53487500
1	6	1090	CONTINUA	157	200	733	190025000	0.183486	243512500
8	18	58	CONTINUA	4	8	46	7205000	0.137931	250717500
20	43	65	CONTINUA	18	6	41	4812500	0.092308	255530000
0	5	2145	CONTINUA	124	165	1856	122512500	0.076923	378042500
9	21	13	BAJA+1	11	1	1	742500	0.076923	378785000
21	44	18	BAJA+1	13	1	4	605000	0.055556	379390000
4	12	1161	CONTINUA	52	48	1061	20872500	0.041344	400262500
11	23	31	CONTINUA	9	1	21	247500	0.032258	400510000
13	29	2792	CONTINUA	91	87	2613	18947500	0.031160	419457500
6	15	4590	CONTINUA	91	125	4374	11275000	0.027233	430732500
12	28	3179	CONTINUA	27	52	3100	-30222500	0.016357	400510000
16	35	7428	CONTINUA	80	104	7244	-89870000	0.014001	310640000
2	8	1039	CONTINUA	10	14	1015	-13172500	0.013474	297467500
14	31	9251	CONTINUA	63	65	9123	-182902500	0.007026	114565000
7	16	1765	CONTINUA	7	12	1746	-35337500	0.006799	79227500
17	36	45807	CONTINUA	128	139	45540	-1106792500	0.003034	-1027565000
18	38	82646	CONTINUA	23	55	82568	-2212265000	0.000665	-3239830000
22	45	18	CONTINUA	0	0	18	-495000	0.000000	-3240325000
5	13	4	BAJA+1	3	0	1	-110000	0.000000	-3240435000
3	9	1	BAJA+1	1	0	0	-27500	0.000000	-3240462500
23	46	48	CONTINUA	0	0	48	-1320000	0.000000	-3241782500

Próximos pasos: Generar código con `leaf_df` New interactive sheet

- **¿Cuál es la ganancia máxima que puede obtener con este modelo?**
- **¿Nota alguna relación con la probabilidad de corte óptima discutida en la clase anterior?**

Matrices de Confusión

Todos ya sabemos que es son las matrices de confusión y como usando los valores de la misma, podemos calcular varios estadísticos

Una diferencia a lo trabajado hasta ahora en la maestría, es que por cada modelo no vamos a generar 1 matriz de confusión, sino múltiples, 1 por cada punto de corte.

Para eso tenemos que proceder:

- Agrupar los buenos y los malos, ya que la mayoría de los indicadores son binarios
- Calcular los valores: **TP, TN, FP y FN**

```
leaf_df["evento"] = leaf_df["BAJA+2"]
leaf_df["no_evento"] = leaf_df["CONTINUA"] + leaf_df["BAJA+1"]

leaf_df["evento_acum"] = leaf_df["evento"].cumsum()
leaf_df["no_evento_acum"] = leaf_df["no_evento"].cumsum()

total_evento = leaf_df["evento"].sum()
total_noevento = leaf_df["no_evento"].sum()

leaf_df['evento_restantes'] = total_evento - leaf_df["evento_acum"]
leaf_df['noevento_restantes'] = total_noevento - leaf_df["no_evento_acum"]

leaf_df["TP"] = leaf_df["evento_acum"]
leaf_df["TN"] = leaf_df["noevento_restantes"]
leaf_df["FP"] = leaf_df["no_evento_acum"]
leaf_df["FN"] = leaf_df["evento_restantes"]

leaf_df
```

	Node	Samples	Predicted Class	BAJA+1	BAJA+2	CONTINUA	ganancia	prob_baja_2	gan_acumulada	evento	no_evento	evento_ac
15	32	1	BAJA+2	0	1	0	1072500	1.000000	1072500	1	0	
19	39	2	BAJA+2	0	1	1	1045000	0.500000	2117500	1	1	
10	22	132	BAJA+2	44	50	38	51370000	0.378788	53487500	50	82	
1	6	1090	CONTINUA	157	200	733	190025000	0.183486	243512500	200	890	2
8	18	58	CONTINUA	4	8	46	7205000	0.137931	250717500	8	50	2
20	43	65	CONTINUA	18	6	41	4812500	0.092308	255530000	6	59	2
0	5	2145	CONTINUA	124	165	1856	122512500	0.076923	378042500	165	1980	4
9	21	13	BAJA+1	11	1	1	742500	0.076923	378785000	1	12	4
21	44	18	BAJA+1	13	1	4	605000	0.055556	379390000	1	17	4
4	12	1161	CONTINUA	52	48	1061	20872500	0.041344	400262500	48	1113	4
11	23	31	CONTINUA	9	1	21	247500	0.032258	400510000	1	30	4
13	29	2792	CONTINUA	91	87	2613	18947500	0.031160	419457500	87	2704	5
6	15	4590	CONTINUA	91	125	4374	11275000	0.027233	430732500	125	4465	6
12	28	3179	CONTINUA	27	52	3100	-30222500	0.016357	400510000	52	3127	7
16	35	7428	CONTINUA	80	104	7244	-89870000	0.014001	310640000	104	7324	8
2	8	1039	CONTINUA	10	14	1015	-13172500	0.013474	297467500	14	1025	8
14	31	9251	CONTINUA	63	65	9123	-182902500	0.007026	114565000	65	9186	9
7	16	1765	CONTINUA	7	12	1746	-35337500	0.006799	79227500	12	1753	9
17	36	45807	CONTINUA	128	139	45540	-1106792500	0.003034	-1027565000	139	45668	10
18	38	82646	CONTINUA	23	55	82568	-2212265000	0.000665	-3239830000	55	82591	11
22	45	18	CONTINUA	0	0	18	-495000	0.000000	-3240325000	0	18	11
5	13	4	BAJA+1	3	0	1	-110000	0.000000	-3240435000	0	4	11
3	9	1	BAJA+1	1	0	0	-27500	0.000000	-3240462500	0	1	11
23	46	48	CONTINUA	0	0	48	-1320000	0.000000	-3241782500	0	48	11

Próximos pasos: Generar código con `leaf_df` New interactive sheet

Nos quedamos con una tabla reducida

```
leaf2_df = leaf_df[["prob_baja_2", "TP", "TN", "FP", "FN"]]
leaf2_df.sort_values("prob_baja_2", ascending=False, inplace=True)
leaf2_df
```

```
/tmp/ipykernel_2202/252966762.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  leaf2_df.sort_values("prob_baja_2", ascending=False, inplace=True)
```

	prob_baja_2	TP	TN	FP	FN
15	1.000000	1	162148	0	1134
19	0.500000	2	162147	1	1133
10	0.378788	52	162065	83	1083
1	0.183486	252	161175	973	883
8	0.137931	260	161125	1023	875
20	0.092308	266	161066	1082	869
0	0.076923	431	159086	3062	704
9	0.076923	432	159074	3074	703
21	0.055556	433	159057	3091	702
4	0.041344	481	157944	4204	654
11	0.032258	482	157914	4234	653
13	0.031160	569	155210	6938	566
6	0.027233	694	150745	11403	441
12	0.016357	746	147618	14530	389
16	0.014001	850	140294	21854	285
2	0.013474	864	139269	22879	271
14	0.007026	929	130083	32065	206
7	0.006799	941	128330	33818	194
17	0.003034	1080	82662	79486	55
18	0.000665	1135	71	162077	0
22	0.000000	1135	53	162095	0
5	0.000000	1135	49	162099	0
3	0.000000	1135	48	162100	0
23	0.000000	1135	0	162148	0

Próximos pasos: Generar código con leaf2_df | New interactive sheet

Con esta tabla podemos nos solo calcular los indicadores clásicos, sino que podemos hacer por cada punto de corte, dibujar la curva y establecer cuál es el optimo punto de corte que máximiza el métrica.

curva ROC

La curva ROC (Receiver Operating Characteristic) es una herramienta gráfica utilizada en ciencia de datos para evaluar el rendimiento de un modelo de clasificación binaria. Es particularmente útil para entender el trade-off entre cuánto podemos ganar, en función de cuanto estamos dispuestos a perder.

Se dibuja como **Tasa de Falsos Positivos** contra **Tasa de Verdaderos Positivos**

```
leaf2_df['TPR'] = leaf2_df['TP'] / (leaf2_df['TP'] + leaf2_df['FN'])
leaf2_df['FPR'] = leaf2_df['FP'] / (leaf2_df['FP'] + leaf2_df['TN'])

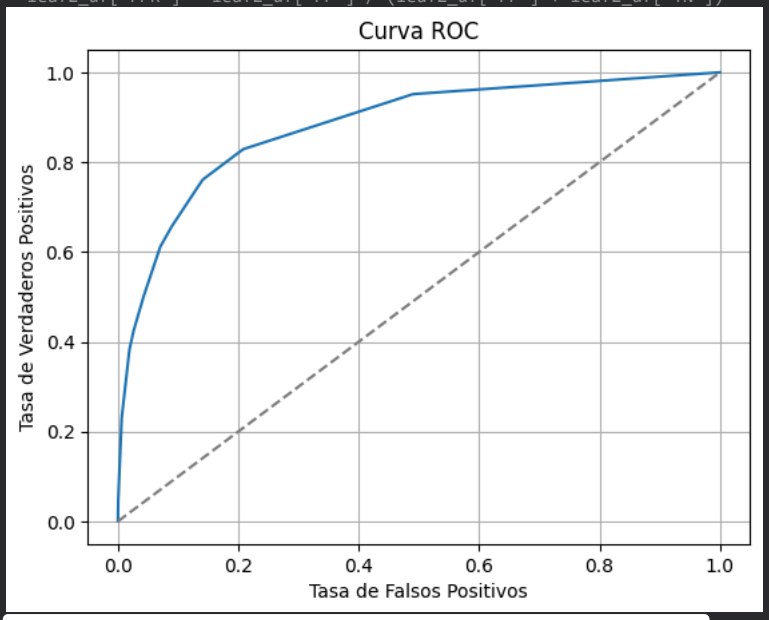
sns.lineplot(x='FPR', y='TPR', data=leaf2_df)
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curva ROC')
plt.grid(True)
plt.show()

# Nota: Es probable que los valores de FPR y TPR esten lejos del [0,0] y [1,1]
# si ese es su caso, debe agregarlos a mano a la tabla
```

```
/tmp/ipykernel_2202/2773130844.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  leaf2_df['TPR'] = leaf2_df['TP'] / (leaf2_df['TP'] + leaf2_df['FN'])
/tmp/ipykernel_2202/2773130844.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  leaf2_df['FPR'] = leaf2_df['FP'] / (leaf2_df['FP'] + leaf2_df['TN'])
```



- La línea de identidad indica el comportamiento de un clasificador que no tiene ninguna habilidad para distinguir entre las clases positivas y negativas.
- Es una métrica independiente de la distribución de clases: La curva ROC no depende de la distribución de las clases en los datos (balance de clases), lo que la hace útil en situaciones donde las clases están desbalanceadas.
- Es una evaluación global del modelo: La curva ROC proporciona una visión más completa del rendimiento del modelo.

No todo son rosas, si queremos comparar 2 modelos, mirar curvas puede ser complejo. Por esto mismo, se suele calcular la métrica **AUC** (Area Under the Curve) como un estatístico de la calidad del modelo.

```
np.trapz(leaf2_df['TPR'], leaf2_df['FPR']) # calcula el área de bajo de la curva
```

```
/tmp/ipykernel_2202/3154154329.py:1: DeprecationWarning: `trapz` is deprecated. Use `trapezoid` instead, or one of the numer
  np.trapz(leaf2_df['TPR'], leaf2_df['FPR']) # calcula el área de bajo de la curva
np.float64(0.881222511788056)
```

- AUC = 1: El modelo clasifica perfectamente todas las instancias. Más cerca de 1 se encuentra el indicador, mejor.
- AUC = 0.5: El modelo no tiene capacidad discriminativa; su rendimiento es equivalente a realizar una clasificación aleatoria.
- AUC < 0.5: Indica un rendimiento peor que una clasificación aleatoria (dale vuelta a la clase).
- Más de una curva puede tener la misma AUC, se suele elegir a la más empinada

Tarea

- Cambie los parámetros del modelo y vea los impactos en cada una de las métricas.
- ¿Cuál fue el modelo que dió más ganancia económica? ¿El peor?
- ¿Cuál fue el modelo que dió mayor AUC? ¿El peor?
- Calcule, dibuje y encuentre el punto de corte óptimo para:
- Accuracy
- Sensibilidad
- Especificidad
- F1-Score

EDA

Sumarizando

Empezamos por un pantallazo sobre todas las variables, con una **sumarización**, agregando 2 estadísticos muy importantes que no trae por defecto **Pandas**

- **Número de valores únicos de una variables**: Muchas veces variables categóricas están codificadas en variables numéricas. Este indicador nos ayudará a detectarlas
- **Número de missing values**: Que cantidad de registros tienen valores numéricos y cuales tiene faltantes. El tratamiento (o no) de missings es uno de los temas que navegarán en los **experimentos colectivos**

Navegue la tabla a la par de el **diccionario de datos**. Los alumnos tienen a subestimar el poder de entender el conjunto de datos a la hora de hacer un buen modelo.

```
desc_stats = data.describe(include='all') # Calcula estadísticas descriptivas básicas
missing_values = data.isnull().sum() # Calcula el número de valores faltantes por columna
unique_values = data.nunique() # Calcula el número de valores distintos por columna

# Combina las estadísticas en un solo DataFrame
summary_table = pd.DataFrame({
    'Missing Values': missing_values,
    'Unique Values': unique_values
})

pd.concat([summary_table.transpose(), desc_stats], axis=0).transpose()
```

	Missing Values	Unique Values	count	unique	top	freq	mean	std	min
numero_de_cliente	0.0	163284.0	163284.0	NaN	NaN	NaN	37665648.152238	15780906.293389	12159854.0
foto_mes	0.0	1.0	163284.0	NaN	NaN	NaN	202104.0	0.0	202104.0
active_quarter	0.0	2.0	163284.0	NaN	NaN	NaN	0.986876	0.113808	0.0
cliente_vip	0.0	2.0	163284.0	NaN	NaN	NaN	0.002609	0.051011	0.0
internet	0.0	5.0	163284.0	NaN	NaN	NaN	0.06237	0.313295	0.0
...	...	...	...	...	...	...	...	...	...
Visa_mconsumototal	21790.0	115641.0	141494.0	NaN	NaN	NaN	21864.234922	40331.363608	-60767.61
Visa_cconsumos	21790.0	132.0	141494.0	NaN	NaN	NaN	8.160565	9.864165	0.0
Visa_cadelantosefectivo	21790.0	14.0	141494.0	NaN	NaN	NaN	0.008792	0.201397	0.0
Visa_mpagominimo	7987.0	5726.0	155297.0	NaN	NaN	NaN	6082.537715	18668.150809	0.0
clase_ternaria	0	3	163284	3	CONTINUA	161192	NaN	NaN	NaN

155 rows × 13 columns

Explorando las variables más importantes del modelo

Vamos a profundizar sobre las variables utilizadas por el modelo. Si las eligió el modelo, seguro nos ayudará a entender un poco más los datos, no?

Para este fin, vamos a obtener la importancia de variable del modelo

```
feature_importances = pd.DataFrame({'feature': X.columns, 'importance': model.feature_importances_})
feature_importances.sort_values('importance', ascending=False)
```

	feature	importance
107	ctrx_quarter	0.324508
111	Master_status	0.170410
22	mcuentas_saldo	0.141978
33	mprestamos_personales	0.097128
113	Master_Fvencimiento	0.093749
...	...	...
148	Visa_mpagosdolares	0.000000
150	Visa_mconsumototal	0.000000
151	Visa_cconsumos	0.000000
152	Visa_cadelantosefectivo	0.000000
153	Visa_mpagominimo	0.000000

154 rows × 2 columns

Tomaremos algunas para hacer un doble click

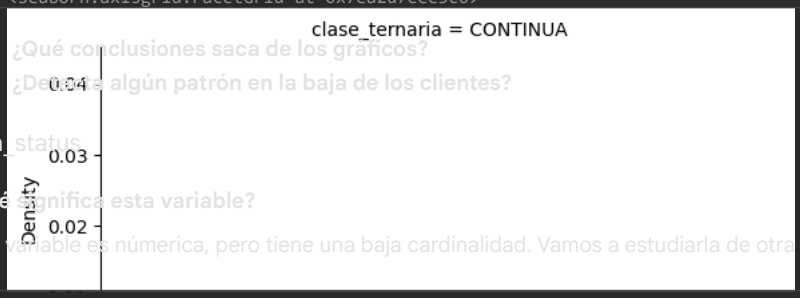
ctrx_quarter

¿Qué significa esta variable?

Cómo se trata de una variable numéricas, vamos a graficar la densidad por cada clase:

```
g = sns.FacetGrid(data, row="clase_ternaria", height=3, aspect=2)
g.map(sns.histplot, "ctrx_quarter", stat='density')
```

```
<seaborn.axisgrid.FacetGrid at 0x7ea2d7eee3c0>
```



- ¿Qué conclusiones saca de los gráficos?
- ¿Detecta algún patrón en la baja de los clientes?

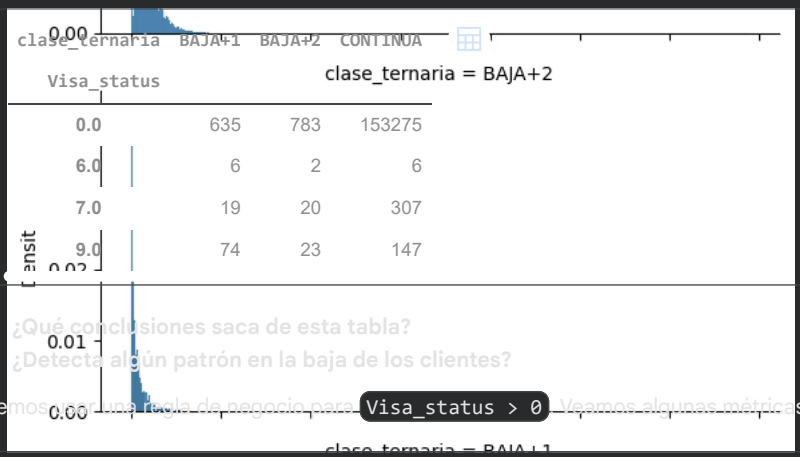
Visa_status

¿Qué significa esta variable?

Esta variable es numerica, pero tiene una baja cardinalidad. Vamos a estudiarla de otra manera

```
pd.crosstab(data['Visa_status'], data['clase_ternaria'])
```

clase_ternaria BAJA+1 BAJA+2 CONTINUA



Visa_status			
0.0	635	783	153275
6.0	6	2	6
7.0	19	20	307
9.0	74	23	147

- ¿Qué conclusiones saca de esta tabla?
- ¿Detecta algún patrón en la baja de los clientes?

Podemos usar una regla de negocio para `Visa_status > 0`. Veamos algunas métricas de esta misma:

```
numero_clientes = data.shape[0]
cantidad_baja2 = data[data['clase_ternaria'] == 'BAJA+2'].shape[0]

cantidad_visa_gt_0 = data[data['Visa_status'] > 0].shape[0]
cantidad_visa_gt_0_and_baja2 = data[(data['Visa_status'] > 0) & (data['clase_ternaria'] == 'BAJA+2')].shape[0]

P_baja2 = cantidad_baja2 / numero_clientes

P_visa_status_gt_0_and_baja2 = (cantidad_visa_gt_0_and_baja2 / cantidad_visa_gt_0)

lift = P_visa_status_gt_0_and_baja2 / P_baja2

columnas = ["Métrica", "Valor"]
registros = [
    ("Total de clientes", f"{numero_clientes}"),
    ("P(Visa_status > 0 y BAJA+2)", f"{P_visa_status_gt_0_and_baja2:.6f}"),
    ("P(BAJA+2)", f"{P_baja2:.6f}"),
    ("Lift", f"{lift:.6f}")
]

pd.DataFrame(registros, columns=columnas)
```